Python Activity-Based Learning

10 Selected Case Studies Solutions

Name: Jalluri Vaishnavi

Section: CSE 1

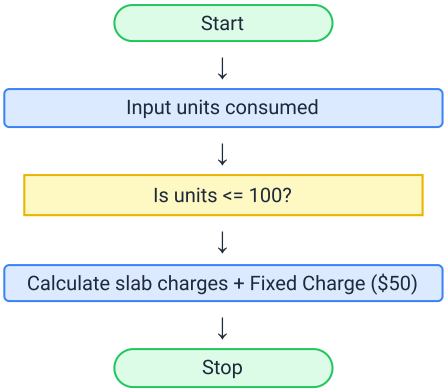
Roll No: 26000024

1. Progressive Electricity Billing (Case Study 2)

1. PROBLEM STATEMENT

Calculate household electricity bills using multiple consumption slabs, fixed charges, and surcharges.

2. FLOWCHART



3. PYTHON SOURCE CODE

```
units = float(input("Enter total units consumed: ")) fixed_charge = 50.0 if units <= 100: bill = u
```

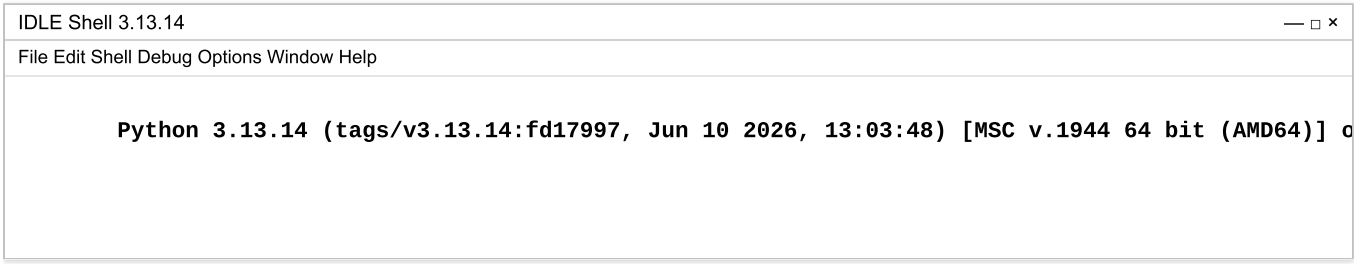
4. SAMPLE INPUT AND OUTPUT

```
Input: 250 units Output: Total Electricity Bill: $650.00
```

5. CONCLUSION

Successfully computed multi-slab billing with fixed surcharge addition.

6. OUTPUT SCREENSHOT



1. Progressive Electricity Billing (Contd.)

7. OBJECTIVES

- Apply conditional statements (if-elif-else) for multi-tier slab calculations.
- Implement numerical arithmetic operations and print formatted outputs.

8. ASSUMPTIONS

- Minimum fixed meter charge is $50.
- Slab rates: $1.5/unit (<=100), $3.0/unit (101-300), $5.0/unit (>300).

9. ALGORITHM

1. START
2. Read units consumed.
3. If units <= 100, bill = units * 1.5
4. Else if units <= 300, bill = 150 + (units - 100) * 3.0
5. Else, bill = 150 + 600 + (units - 300) * 5.0
6. Add fixed charge of 50 to bill.
7. Display total bill amount. STOP

10. PSEUDOCODE

```
BEGIN READ units SET fixed_charge = 50.0 IF units <= 100 THEN SET bill = units * 1.5 ELSE IF units
```

11. TEST CASES

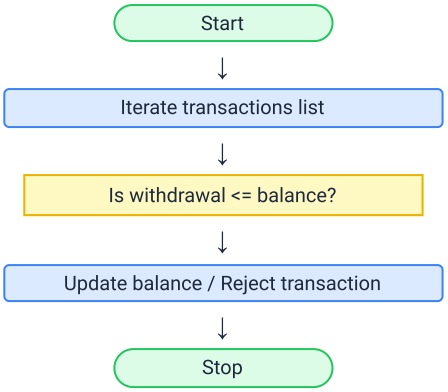
- Test Case 1: units = 80 -> Expected Bill: $170.00
- Test Case 2: units = 250 -> Expected Bill: $650.00
- Test Case 3: units = 350 -> Expected Bill: $1050.00

2. ATM Transaction Simulation (Case Study 13)

1. PROBLEM STATEMENT

Process deposits and withdrawals, update balance, and enforce withdrawal limits.

2. FLOWCHART



3. PYTHON SOURCE CODE

```
balance = 1000.0 # Initial balance transactions = [('D', 500), ('W', 200), ('W', 1500), ('D', 100)
```

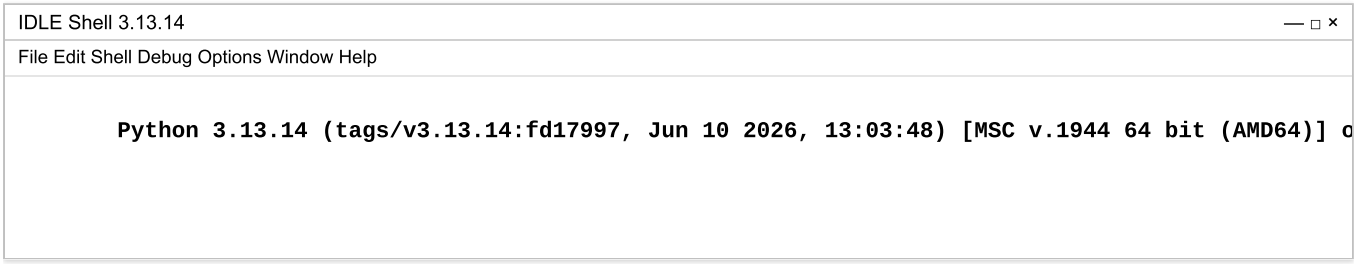
4. SAMPLE INPUT AND OUTPUT

```
Deposited: $500. New Balance: $1500.0 Withdrew: $200. New Balance: $1300.0 Withdrawal of $1500 rej
```

5. CONCLUSION

Simulated real-time banking balance updates while preventing overdrafts.

6. OUTPUT SCREENSHOT



2. ATM Transaction Simulation (Contd.)

7. OBJECTIVES

• Utilize list of tuples and unpacking for batch transaction execution.

8. ASSUMPTIONS

• Starting initial balance is $1000. 'D' stands for Deposit, 'W' for Withdrawal.

9. ALGORITHM

1. START
2. Initialize balance = 1000.
3. For each tuple in transaction list:
4. If type is 'D', add amount to balance.
5. If type is 'W' and amount <= balance, subtract amount from balance.
6. Otherwise display error message.
7. STOP

10. PSEUDOCODE

```
BEGIN SET balance = 1000.0 FOR EACH (type, amount) IN transaction_list DO IF type == 'D' THEN bala
```

11. TEST CASES

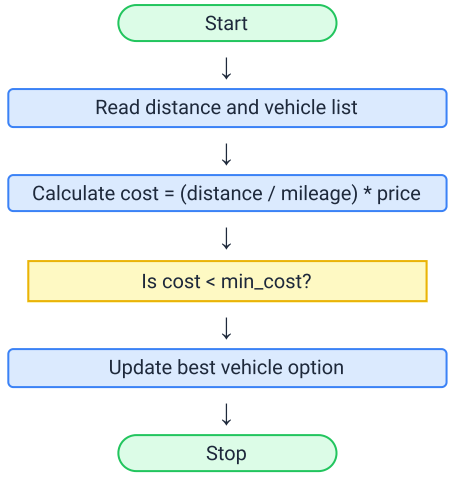
• ('D', 500) -> Balance: $1500
• ('W', 1500) -> Rejected due to insufficient balance

3. Fuel Consumption & Route Comparison (Case Study 18)

1. PROBLEM STATEMENT

Calculate fuel requirements and journey costs for vehicles using distance, mileage, and fuel price.

2. FLOWCHART



3. PYTHON SOURCE CODE

```
distance = 300 # in km vehicles = [("Car A", 15, 100), ("Car B", 20, 100), ("Bike", 40, 100)] min_
```

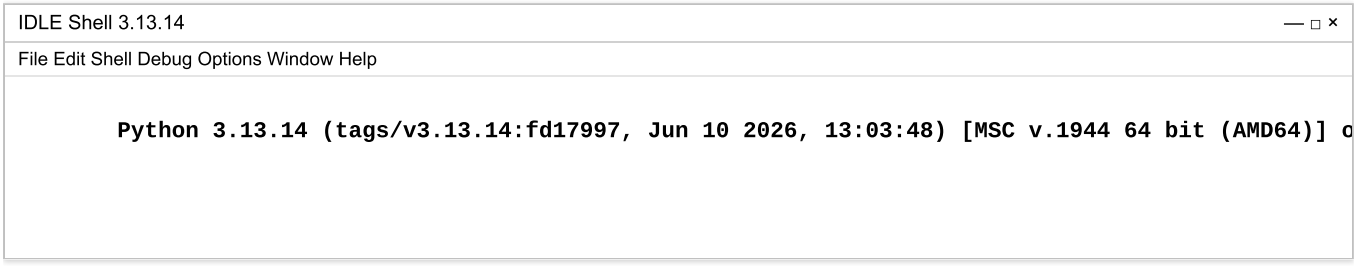
4. SAMPLE INPUT AND OUTPUT

```
Car A Cost: $2000.00 Car B Cost: $1500.00 Bike Cost: $750.00 Lowest-cost feasible option: Bike wit
```

5. CONCLUSION

Identified the most economical vehicle option for a given route distance.

6. OUTPUT SCREENSHOT



3. Fuel Consumption & Route Comparison (Contd.)

7. OBJECTIVES

• Compare multiple options stored in lists and identify minimum values.

8. ASSUMPTIONS

• Fixed distance = 300 km. Mileage is measured in km per liter.

9. ALGORITHM

1. START
2. Set distance = 300, min_cost = infinity.
3. Loop through vehicle records.
4. Compute fuel_needed = distance / mileage.
5. Compute cost = fuel_needed * price.
6. If cost < min_cost, update min_cost and best_vehicle.
7. Print the cheapest vehicle name and cost. STOP

10. PSEUDOCODE

```
BEGIN SET distance = 300 SET min_cost = INFINITY FOR EACH (name, mileage, price) IN vehicles DO SE
```

11. TEST CASES

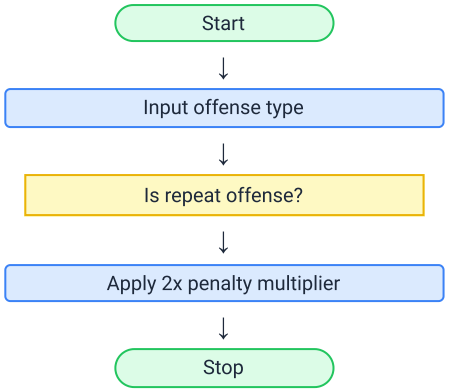
• Distance = 300km -> Bike ($750.00) selected as minimum cost.

4. Traffic Violation Penalty Calculator (Case Study 21)

1. PROBLEM STATEMENT

Calculate final traffic penalties based on offense type and repeat-offense status.

2. FLOWCHART



3. PYTHON SOURCE CODE

```
offense = "speeding" # options: speeding, parking, red_light is_repeat = True base_fine = 0 if off
```

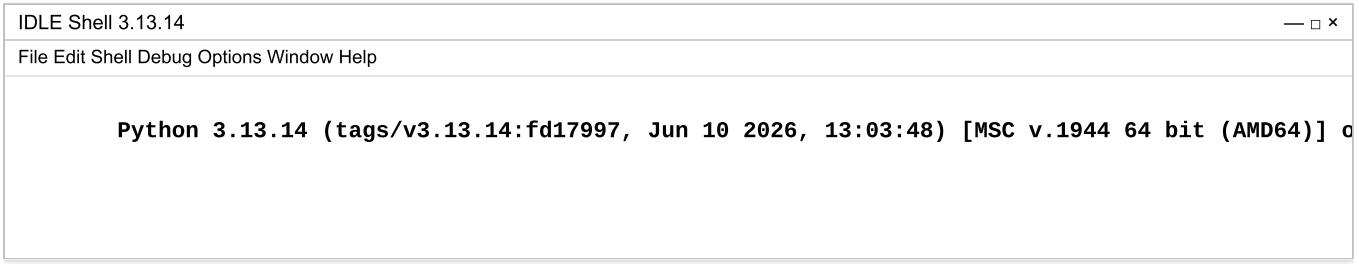
4. SAMPLE INPUT AND OUTPUT

```
Final Penalty for speeding (Repeat: True): $200
```

5. CONCLUSION

Computed total fine after evaluating violation type and repeat offense status.

6. OUTPUT SCREENSHOT



4. Traffic Violation Penalty Calculator (Contd.)

7. OBJECTIVES

- Practice string comparisons and boolean conditions for logic evaluation.

8. ASSUMPTIONS

- Standard base fines: Parking=$50, Speeding=$100, Red Light=$150. Repeat doubles fine.

9. ALGORITHM

1. START
2. Read offense string and is_repeat boolean.
3. Set base_fine according to offense type.
4. If is_repeat is True, total = base_fine * 2.
5. Else total = base_fine.
6. Output total penalty. STOP

10. PSEUDOCODE

```
BEGIN READ offense, is_repeat IF offense == "parking" THEN base_fine = 50 ELSE IF offense == "spee
```

11. TEST CASES

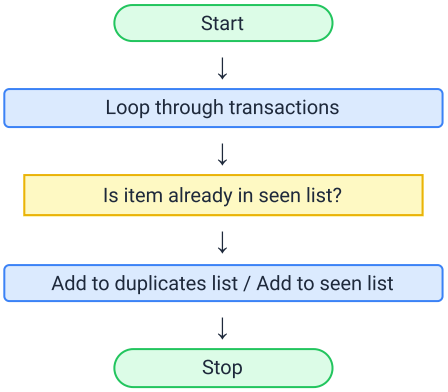
- Speeding, Repeat=True -> $200
- Parking, Repeat=False -> $50

5. Repeated Transaction Detection (Case Study 28)

1. PROBLEM STATEMENT

Process transaction IDs and amounts in a list to detect duplicate entries.

2. FLOWCHART



3. PYTHON SOURCE CODE

```
transactions = [101, 102, 103, 102, 104, 101] seen = [] duplicates = [] for txn in transactions: i
```

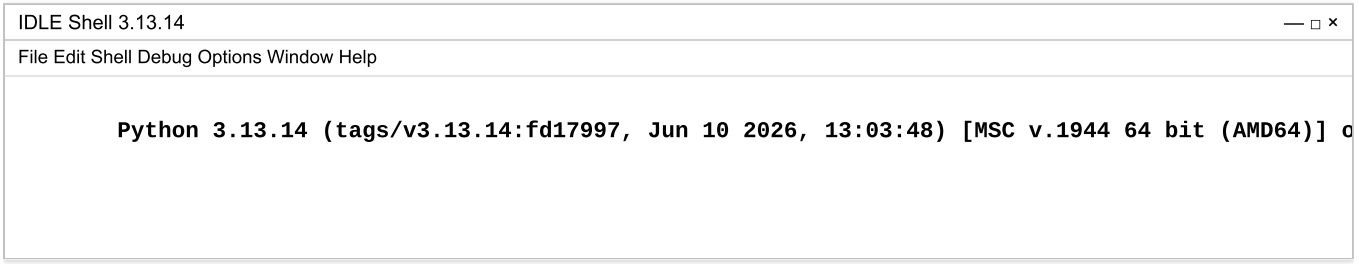
4. SAMPLE INPUT AND OUTPUT

```
All Transactions: [101, 102, 103, 102, 104, 101] Detected Duplicate Transaction IDs: [102, 101]
```

5. CONCLUSION

Identified duplicate entries in transaction logs using list operations.

6. OUTPUT SCREENSHOT



5. Repeated Transaction Detection (Contd.)

7. OBJECTIVES

• Use membership operators (`in`, `not in`) and list `.append()` methods.

8. ASSUMPTIONS

• Transaction IDs are integers. Duplicate values flag suspicious entries.

9. ALGORITHM

1. START
2. Initialize transactions list, empty seen list, empty duplicates list.
3. For each txn in transactions:
4. If txn in seen and not in duplicates, append to duplicates.
5. Else if txn not in seen, append to seen.
6. Output duplicates list. STOP

10. PSEUDOCODE

```
BEGIN SET transactions = [101, 102, 103, 102, 104, 101] SET seen = [] SET duplicates = [] FOR EACH
```

11. TEST CASES

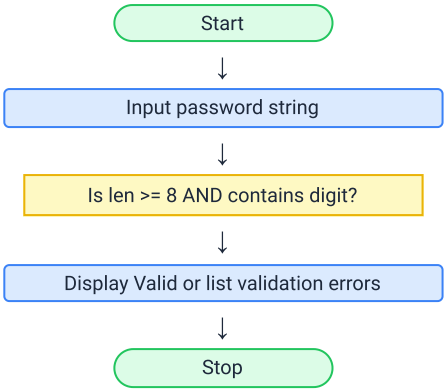
• List [101, 102, 103, 102, 104, 101] -> Output [102, 101]

6. Password Policy Validation Engine (Case Study 29)

1. PROBLEM STATEMENT

Evaluate sample password strings against length and digit requirements.

2. FLOWCHART



3. PYTHON SOURCE CODE

```
password = "Secure123" errors = [] if len(password) < 8: errors.append("Password must be at least
```

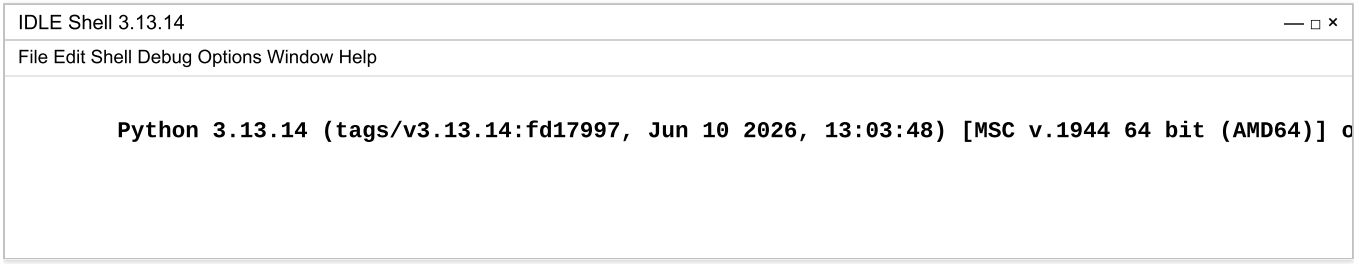
4. SAMPLE INPUT AND OUTPUT

```
Input: "Secure123" Output: Password is valid!
```

5. CONCLUSION

Successfully evaluated password security rules using string functions.

6. OUTPUT SCREENSHOT



6. Password Policy Validation Engine (Contd.)

7. OBJECTIVES

• Use string functions (`len()`, `.isdigit()`) to validate user inputs.

8. ASSUMPTIONS

• Minimum length required is 8 characters. Password must contain at least one digit.

9. ALGORITHM

1. START
2. Read password string.
3. Check if len(password) < 8. If so, append error message.
4. Check if password contains a digit. If not, append error message.
5. If error list is empty, display success, else display errors. STOP

10. PSEUDOCODE

```
BEGIN READ password SET errors = [] IF LENGTH(password) < 8 THEN APPEND "Min length 8 required" TO
```

11. TEST CASES

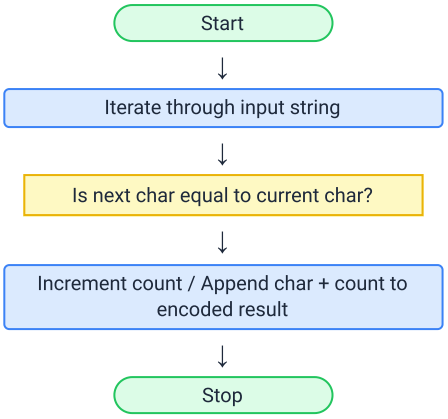
• "Secure123" -> Valid
• "sec" -> Fails length and digit requirements

7. Text Compression Using Character Runs (Case Study 35)

1. PROBLEM STATEMENT

Encode a string by grouping consecutive repeated characters (Run-Length Encoding).

2. FLOWCHART



3. PYTHON SOURCE CODE

```
text = "aaabbcccca" encoded = "" i = 0 while i < len(text): count = 1 while i + 1 < len(text) and
```

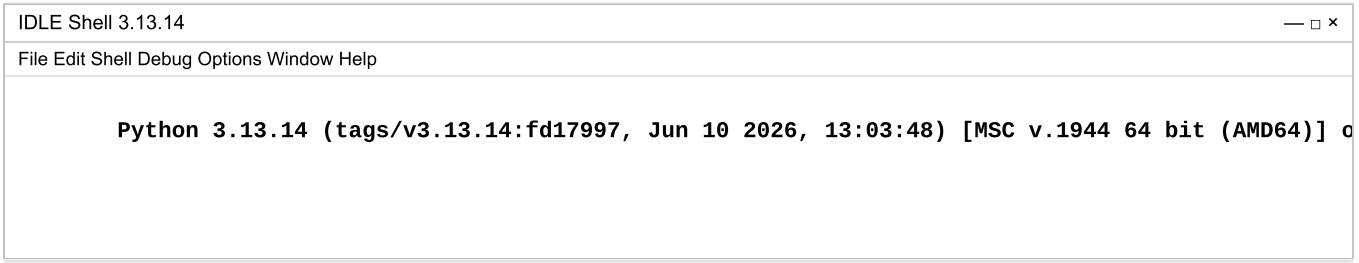
4. SAMPLE INPUT AND OUTPUT

```
Original String: aaabbcccca Encoded String: a3b2c4a1
```

5. CONCLUSION

Successfully compressed repetitive consecutive characters into run-length notation.

6. OUTPUT SCREENSHOT



7. Text Compression Using Character Runs (Contd.)

7. OBJECTIVES

- Utilize nested while-loops for string traversal and basic text compression algorithms.

8. ASSUMPTIONS

- Input string consists of lowercase English alphabet characters.

9. ALGORITHM

1. START
2. Set i = 0, empty string encoded.
3. While i < length of text:
4. Set count = 1.
5. While text[i] == text[i+1], increment count and i.
6. Append text[i] + str(count) to encoded.
7. Increment i. Output encoded text. STOP

10. PSEUDOCODE

```
BEGIN SET text = "aaabbcccca" SET encoded = "" SET i = 0 WHILE i < LENGTH(text) DO SET count = 1 W
```

11. TEST CASES

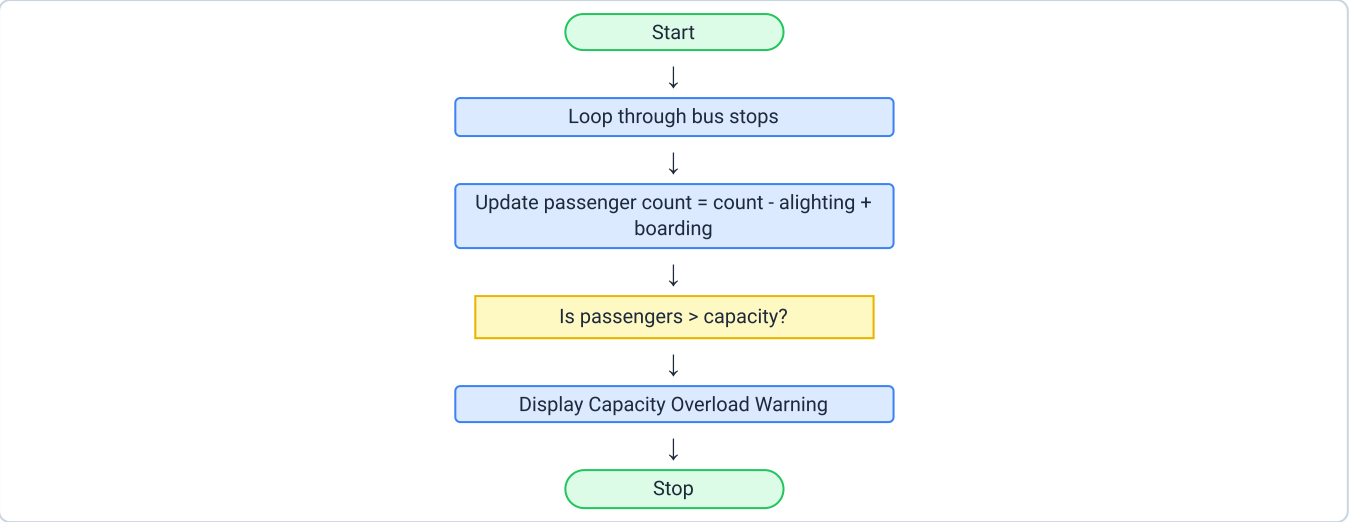
- "aaabbcccca" -> "a3b2c4a1"

8. Bus Passenger Occupancy Simulation (Case Study 51)

1. PROBLEM STATEMENT

Process boarding and alighting counts across stops and flag capacity violations.

2. FLOWCHART



3. PYTHON SOURCE CODE

```
capacity = 50 current_passengers = 10 stops = [(15, 2), (10, 5), (5, 12), (20, 5)] for i, (boardin
```

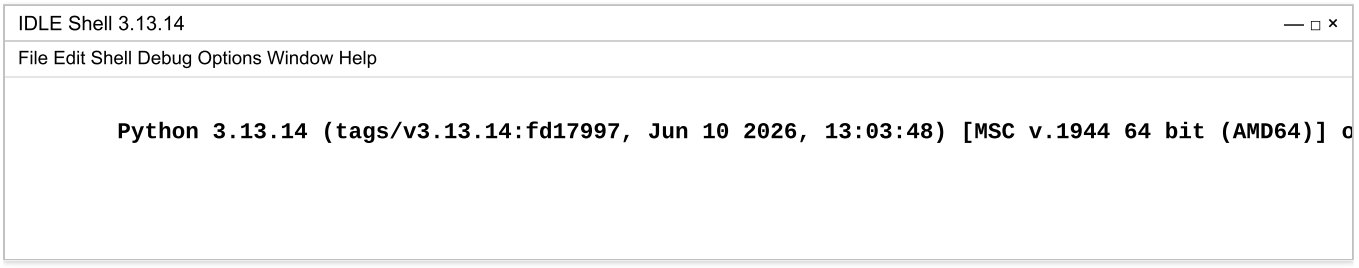
4. SAMPLE INPUT AND OUTPUT

```
Stop 1: Passengers on board = 23 Stop 2: Passengers on board = 28 Stop 3: Passengers on board = 21
```

5. CONCLUSION

Monitored live bus occupancy across stops and validated bus seating limits.

6. OUTPUT SCREENSHOT



8. Bus Passenger Occupancy Simulation (Contd.)

7. OBJECTIVES

• Apply `enumerate()` loops and tuple unpacking for sequence tracking.

8. ASSUMPTIONS

• Bus capacity limit = 50 passengers. Alighting happens before boarding at stops.

9. ALGORITHM

1. START
2. Set capacity = 50, current_passengers = 10.
3. For each stop (boarding, alighting):
4. current_passengers = current_passengers - alighting + boarding.
5. If current_passengers > capacity, display warning.
6. Output updated stop details. STOP

10. PSEUDOCODE

```
BEGIN SET capacity = 50 SET current_passengers = 10 FOR EACH (boarding, alighting) AT stop DO SET
```

11. TEST CASES

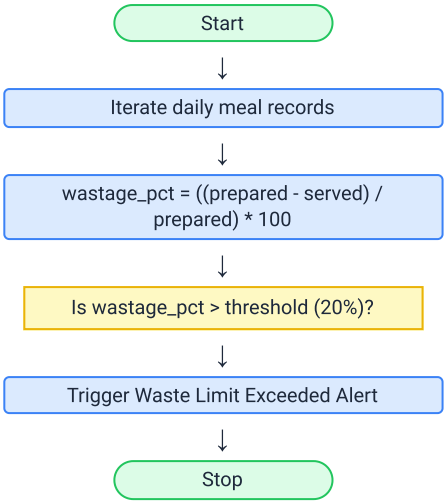
• Initial=10, Stop 1 (+15, -2) -> 23 passengers on board.

9. Food Waste Monitoring System (Case Study 53)

1. PROBLEM STATEMENT

Compare prepared meals and meals served to calculate wastage percentages.

2. FLOWCHART



3. PYTHON SOURCE CODE

```
weekly_data = [(100, 85), (120, 90), (110, 105), (95, 70), (130, 100)] threshold_percentage = 20.0
```

4. SAMPLE INPUT AND OUTPUT

```
Day 1: Wastage = 15 meals (15.0%) Day 2: Wastage = 30 meals (25.0%) -> Alert: Day 2 exceeded accep
```

5. CONCLUSION

Analyzed daily food wastage percentages and flagged threshold limit violations.

6. OUTPUT SCREENSHOT

IDLE Shell 3.13.14 — □ ×

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] o
```

9. Food Waste Monitoring System (Contd.)

7. OBJECTIVES

- Practice basic percentage calculations and condition monitoring loops.

8. ASSUMPTIONS

- Allowable threshold waste limit is 20.0% of total prepared meals.

9. ALGORITHM

1. START
2. Set threshold = 20.0%.
3. For each day data (prepared, served):
4. Calculate wastage = prepared - served.
5. Calculate wastage_pct = (wastage / prepared) * 100.
6. If wastage_pct > threshold, output alert message.
7. STOP

10. PSEUDOCODE

```
BEGIN SET threshold = 20.0 FOR EACH (prepared, served) IN weekly_data DO SET wastage = prepared -
```

11. TEST CASES

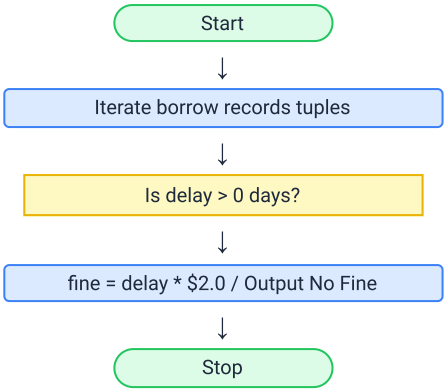
- Day 1 (100, 85) -> 15.0% (Normal)
- Day 2 (120, 90) -> 25.0% (Exceeds Threshold Alert)

10. Library Borrowing & Fine Reconciliation (Case Study 58)

1. PROBLEM STATEMENT

Calculate fines for book return delays using tuple records.

2. FLOWCHART



3. PYTHON SOURCE CODE

```
borrow_records = (("B101", 0), ("B102", 3), ("B103", 10), ("B104", 1)) fine_per_day = 2.0 for book
```

4. SAMPLE INPUT AND OUTPUT

```
Book B101: Returned on time. No fine. Book B102: Delayed by 3 days. Fine: $6.00 Book B103: Delayed
```

5. CONCLUSION

Successfully audited overdue book records and calculated accurate late fines.

6. OUTPUT SCREENSHOT

